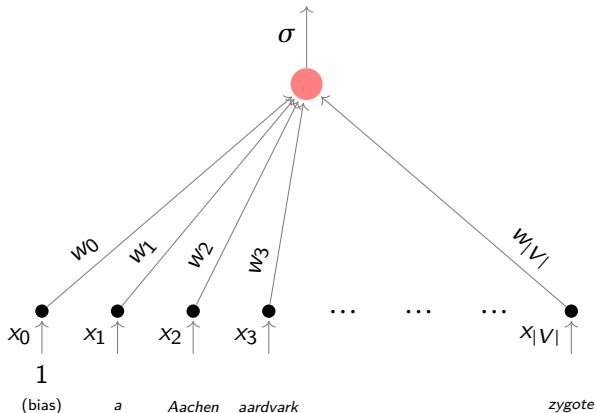


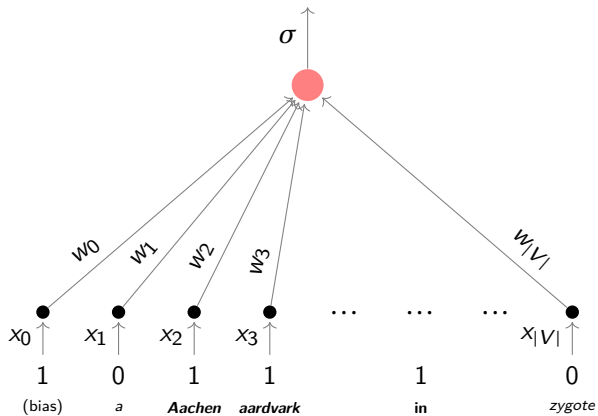
logistic regression on texts

$$p(c|\vec{x}) = \sigma(w_0 + w_1x_1 + w_2x_2 + \cdots + w_{|V|}x_{|V|}) = \sigma(\sum_{i=0}^{|V|} w_i x_i) = \sigma(\vec{w}\vec{x})$$



logistic regression on texts (cntd.)

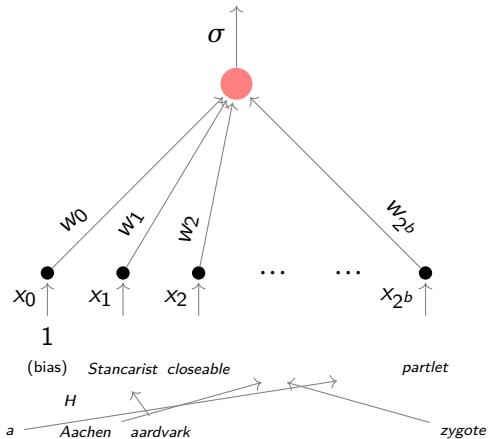
let's classify *aardvark in Aachen*



with hashing trick

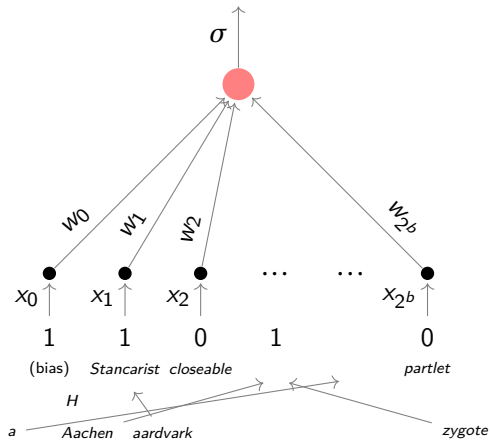
$$p(c|\vec{x}) = \sigma(w_0 + w_1x_1 + w_2x_2 + \cdots + w_{2^b}x_{2^b}) = \sigma(\sum_{i=0}^{2^b} w_i x_i) = \sigma(\vec{w}\vec{x})$$

hashing function $H: V \rightarrow \{1, \dots, 2^b\}$, e.g. MurmurHash3



with hashing trick (cntd.)

aardvark in Aachen the same as Stancarist in Aachen...



with hashing trick you can handle **billions** of features

- words

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)
- word n-grams (bigrams, trigrams)

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)
- word n-grams (bigrams, trigrams)
- character n-grams, prefixes, suffixes

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)
- word n-grams (bigrams, trigrams)
- character n-grams, prefixes, suffixes
- word features (e.g. part-of-speech tags)

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)
- word n-grams (bigrams, trigrams)
- character n-grams, prefixes, suffixes
- word features (e.g. part-of-speech tags)
- text features (e.g. length, average word length, etc.)

with hashing trick you can handle **billions** of features

- words
- words normalized (lemmatized, stemmed, lowercased, etc.)
- word n-grams (bigrams, trigrams)
- character n-grams, prefixes, suffixes
- word features (e.g. part-of-speech tags)
- text features (e.g. length, average word length, etc.)

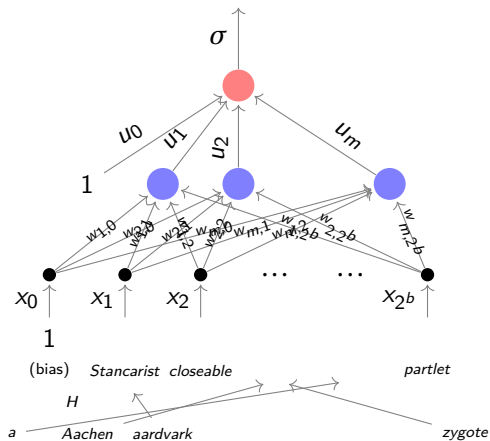
And no need to precompute a vocabulary!

Implementations:

- VowpalWabbit (-b option)
- HashingVectorizer in scikit-learn

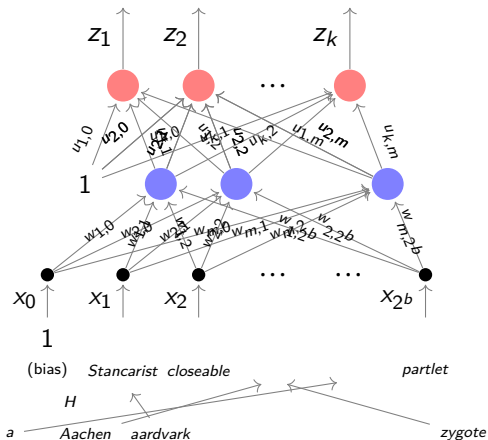
let's make a small neural network

(-nn in Vowpal Wabbit)



what about multi-class classification?

let's say we have $k > 2$ classes



what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$
 - $\sum_i s(z_i) = 1$

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$
 - $\sum_i s(z_i) = 1$
 - $z_i > z_j \Rightarrow s(z_i) > s(z_j)$

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$
 - $\sum_i s(z_i) = 1$
 - $z_i > z_j \Rightarrow s(z_i) > s(z_j)$
- **wrong** solution:

$$s(z_i) = \frac{z_i}{\sum_{j=1}^k z_j}$$

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$
 - $\sum_i s(z_i) = 1$
 - $z_i > z_j \Rightarrow s(z_i) > s(z_j)$
- **wrong** solution:

$$s(z_i) = \frac{z_i}{\sum_{j=1}^k z_j}$$

- right solution: softmax

$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

what about multi-class classification? (cntd.)

- but $z_1, \dots, z_k$ is not a probability distribution
- σ is not the answer (why?)
- we need a function $s: R^k \rightarrow [0, 1]^k$, i.e. $s([z_1, \dots, z_k]) = [p_1, \dots, p_n]$ such that:
 - $s(z_i) = s_i(z) \in [0, 1]$
 - $\sum_i s(z_i) = 1$
 - $z_i > z_j \Rightarrow s(z_i) > s(z_j)$

- **wrong** solution:

$$s(z_i) = \frac{z_i}{\sum_{j=1}^k z_j}$$

- right solution: softmax

$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

- should be called softargmax actually